

Trabajo Práctico — Herencia

Sistema de Delivery

Introducción

Una empresa está desarrollando una aplicación de delivery que permite a sus usuarios realizar pedidos a distintos tipos de comercios.

Actualmente la aplicación trabaja con pedidos de comida y pedidos de supermercado, pero se espera que en el futuro puedan incorporarse nuevos tipos de pedidos.

El objetivo de este trabajo práctico es desarrollar un modelo orientado a objetos que permita representar los distintos tipos de pedidos y procesarlos de manera flexible.

Parte 1 — Análisis del problema

La aplicación debe manejar pedidos.

Todo pedido posee:

- número de pedido;
- nombre del cliente;
- dirección de entrega;
- importe de los productos;
- distancia entre el comercio y el domicilio, expresada en kilómetros.

Actualmente existen dos tipos de pedidos.

Pedido de comida

Además de los datos generales, posee:

- nombre del restaurante;
- cantidad de productos.

Pedido de supermercado

Además de los datos generales, posee:

- nombre del supermercado;
- cantidad de productos.

Consigna

Analizar el problema y diseñar una solución orientada a objetos.

Antes de comenzar a programar, realizar un diagrama que represente las clases que consideren necesarias y las relaciones entre ellas.

El diseño debe contemplar que en el futuro puedan incorporarse nuevos tipos de pedidos.

Parte 2 — Implementación de los pedidos

Implementar en Python las clases necesarias para representar los pedidos.

Como mínimo, el sistema debe permitir crear pedidos equivalentes a los siguientes:

```
pedido1 = PedidoComida(  
    1001,  
    "Juan",  
    "Av. Siempre Viva 123",  
    15000,  
    4.5,  
    "Burger House",  
    3  
)  
  
pedido2 = PedidoSupermercado(  
    1002,  
    "Ana",  
    "San Martín 456",  
    23000,  
    5,  
    "SuperMax",  
    8  
)
```

Los nombres, valores y datos utilizados son solamente ejemplos.

El diseño debe evitar la duplicación innecesaria de código y atributos.

Parte 3 — Costo de envío

La aplicación necesita calcular el costo de envío de cada pedido.

El cálculo depende del tipo de pedido.

Pedido de comida

El costo de envío es:

$\$1.000 + \$300 \times \text{distancia en kilómetros}$

Pedido de supermercado

El costo de envío es:

$\$1.500 + \$400 \times \text{distancia en kilómetros}$

Por ejemplo, para una distancia de 5 km:

Comida:

$1000 + (5 \times 300) = \2500

Supermercado:

$1500 + (5 \times 400) = \3500

El sistema debe permitir obtener el costo de envío de cualquier pedido.

Por ejemplo:

```
print(pedido1.calcular_envio())  
print(pedido2.calcular_envio())
```

Cada tipo de pedido deberá aplicar las reglas de cálculo que le correspondan.

Parte 4 — Costo total

La aplicación debe permitir conocer cuánto debe pagar finalmente el cliente.

El total se obtiene mediante:

importe de los productos + costo de envío

Por ejemplo:

Importe de productos: \$15.000

Costo de envío: \$2.350

Total: \$17.350

Agregar al sistema el comportamiento necesario para poder realizar:

```
print(pedido1.calcular_total())
```

```
print(pedido2.calcular_total())
```

El cálculo del total debe reutilizar el cálculo del costo de envío.

Parte 5 — Procesamiento de pedidos

La aplicación recibe pedidos de diferentes tipos.

Crear una colección que contenga pedidos de comida y pedidos de supermercado.

Por ejemplo:

```
pedidos = [  
    pedido1,  
    pedido2,  
    # otros pedidos...  
]
```

Desarrollar un programa que recorra todos los pedidos y muestre:

- número de pedido;
- cliente;
- importe;
- costo de envío;
- total.

El recorrido debe poder realizarse sin que el código que procesa los pedidos tenga que conocer específicamente qué tipo de pedido está procesando.

El resultado podría ser similar a:

Pedido: 1001
Cliente: Juan
Importe: \$15000
Envío: \$2350
Total: \$17350

Pedido: 1002
Cliente: Ana
Importe: \$23000
Envío: \$3500
Total: \$26500

Parte 6 — Un nuevo tipo de pedido

La aplicación incorpora ahora un nuevo servicio:

Pedido Express

Un pedido express tiene todos los datos generales de un pedido y además posee:

- nivel de prioridad.

El costo de envío se calcula como:

$\$2.500 + \$500 \times \text{distancia en kilómetros}$

Por ejemplo, para 4 km:

$2500 + (4 \times 500) = \4500

Implementar el nuevo tipo de pedido.

Luego incorporarlo a la colección utilizada anteriormente.

Importante

El código encargado de recorrer y procesar los pedidos debe continuar funcionando sin necesidad de realizar modificaciones específicas para **PedidoExpress**.

Parte 7 — Un último desafío

La empresa quiere incorporar pedidos realizados a farmacias.

Pedido de farmacia

Un pedido de farmacia posee:

- nombre de la farmacia;
- cantidad de productos.

Su costo de envío se calcula de la siguiente manera:

$\$1.800 + \$250 \times \text{distancia en kilómetros}$

Implementar este nuevo tipo de pedido.

Agregarlo a la colección de pedidos y verificar que el sistema pueda procesarlo junto con los demás.

Parte 8 — Extensión libre

Agregar un nuevo tipo de pedido inventado por el grupo.

Puede tratarse, por ejemplo, de:

- tienda de mascotas;
- librería;
- tienda de tecnología;
- florería;
- cualquier otro comercio.

El nuevo pedido debe:

- compartir las características comunes de un pedido;
- tener al menos un dato específico;
- implementar su propia forma de calcular el envío;
- poder incorporarse a la colección existente.

El código encargado de procesar los pedidos no debería requerir modificaciones para incorporar este nuevo tipo.